

Project Report
Gesture Music Controller

By
Chanagun Khunphet
662115006

Submitted to
Prof. Phudinan Singkhamfu

953362 - IoT and Embedded Programming for Software
Engineering 1 Department of Software Engineering. Faculty
of College of Arts, Media and Technology.

Chiangmai university

1. Proposal / Abstract

A touchless Spotify control system using hand gestures with an ESP32-CAM. Python (OpenCV/MediaPipe) detects gestures and sends commands via Spotify Web API (spotipy). Users use open/fist and hand movements to play/pause, next/previous, volume, and seek. On-screen preview reduces mistakes; playback info is cached to reduce API calls. Low cost, easy setup, runs on Windows in a home network.

Problem

- Need to control music without touching devices during work, fitness, or cooking; hands can be busy or dirty.
- Voice control suffers from noise, accent/language issues, and privacy concerns.
- Buttons/touch screens are inconvenient when at a distance or while doing activities.

Objectives

- Build real-time Spotify control with ESP32-CAM hand gestures.
- Map defined gestures to Spotify commands: play/pause, next/previous, volume, seek.
- Show on-screen preview and commit on open-hand to avoid errors.

- Reduce API calls using playback caching.
- Work on Windows with low-cost hardware and simple setup.

Scope

- In-scope: single user, one ESP32-CAM, same Wi-Fi as PC, predefined gestures, Spotify Premium/Web API, PC UI preview.
- Out-of-scope: multi-user simultaneous use, very low-light/complex backgrounds, full playlist management, offline mode, mobile app/cloud, automatic learning of new gestures.

2. Software Requirements Specification (SRS)

Functions

- Ready with two open hands
- Play/Pause with two fists (hold ~0.3 s)
- Next: left open + right fist
- Previous: left fist + right open
- Volume preview: one fist move up/down
- Seek preview: one fist move left/right
- Commit: open hand to send volume/seek
- UI feedback overlays
- Playback cache update every 5 s
- Exit with q; show errors and reconnect

I/O requirements

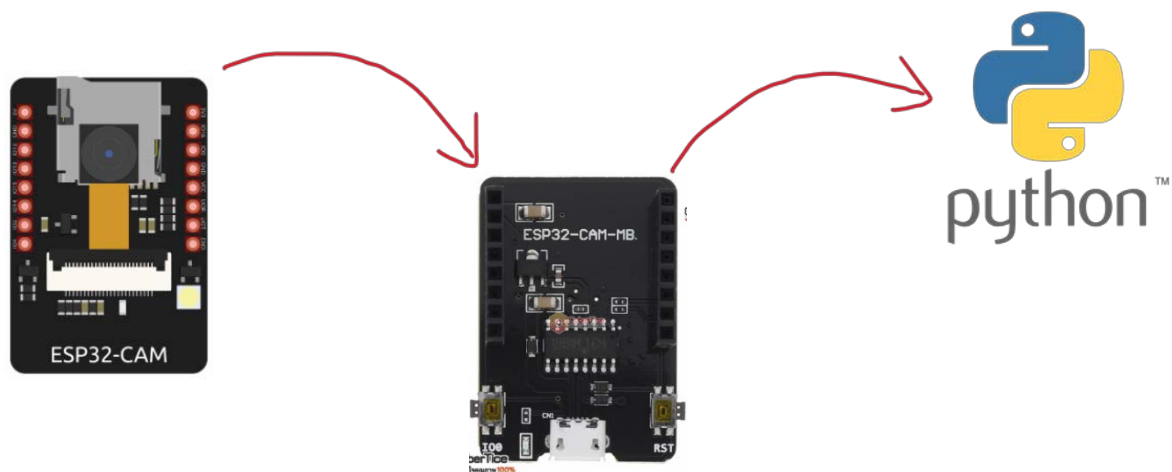
- Inputs: ESP32-CAM MJPEG stream over HTTP; keyboard q; config values; Spotify OAuth tokens
- Outputs: Spotify API commands (play, pause, next, previous, seek ms, volume 0–100); OpenCV window overlays; console logs
- Data: 21 hand landmarks; volume int 0–100; seek ms in [0, duration_ms]; cached playback state

Requirements

- Functional: real-time gesture detection; correct mapping to Spotify; commit only on open hand; debounce; playback caching
- Non-Functional: latency ≤ 500 ms; ≥ 15 FPS; $\geq 90\%$ accuracy in normal light; recover from camera/network errors; protect OAuth data; run on Windows 10/11, Python 3.10+
- Constraints/Assumptions: single user, good lighting, Spotify Premium and internet, same Wi-Fi for ESP32-CAM and PC

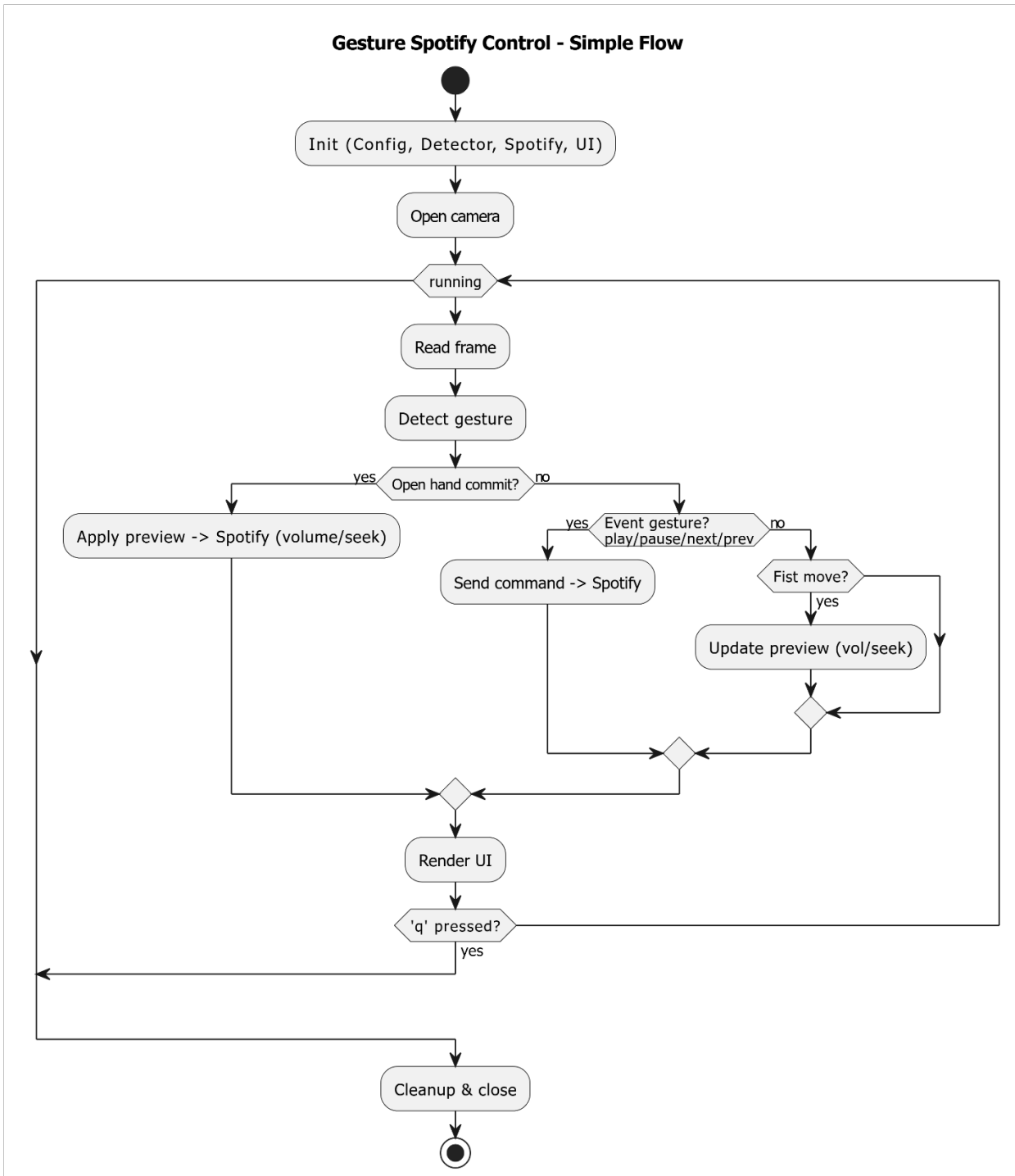
3. Software Design Document (SDD)

Architecture



ESP32 Cam + ESP32 Cam MB + python

Flow Chart



4. Output & Discussion

Ready (two open hands)

- Outcome: Enter ready/preview mode.
- Issues: Hands outside frame or only one detected.
- Mitigation: Hold both hands near center for 0.2–0.3 s with good lighting.

Play/Pause (two fists, hold ~0.3 s)

- Outcome: Toggle play/pause.
- Issues: Gesture too quick; body wobble causes false/ missed trigger.
- Mitigation: Hold 0.3–0.4 s; add ~0.5 s cooldown.

Next (left open + right fist)

- Outcome: Skip to next track.
- Issues: Left/right swapped by mirrored image; overlapping hands confuse detection.
- Mitigation: Ensure image flip matches logic; keep hands apart.

Previous (left fist + right open)

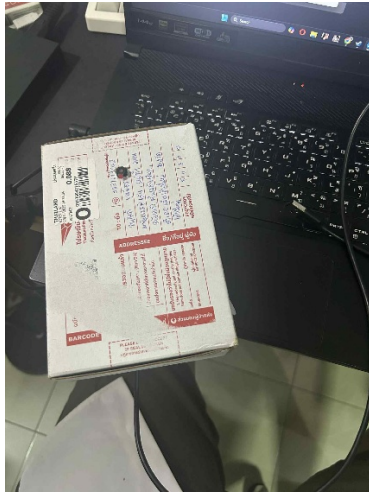
- Outcome: Go to previous track.
- Issues: Same as Next; accidental repeats.

- Mitigation: Edge-trigger plus ~ 0.7 s cooldown.

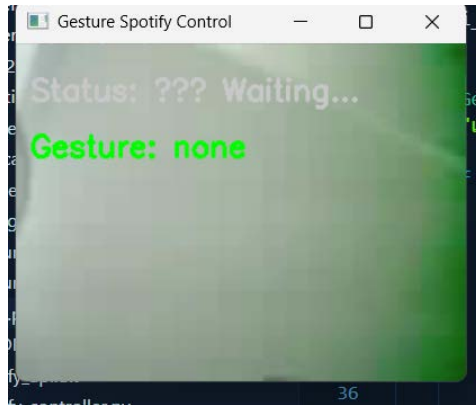
Volume preview (one fist moves up/down)

- Outcome: Preview volume percent (not sent yet).
- Issues: Jitter causes fluctuation; fast movement jumps values.
- Mitigation: Y-axis deadzone, smoothing, 2–3% steps.

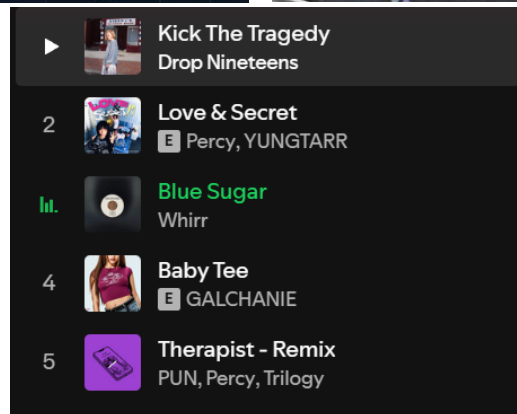
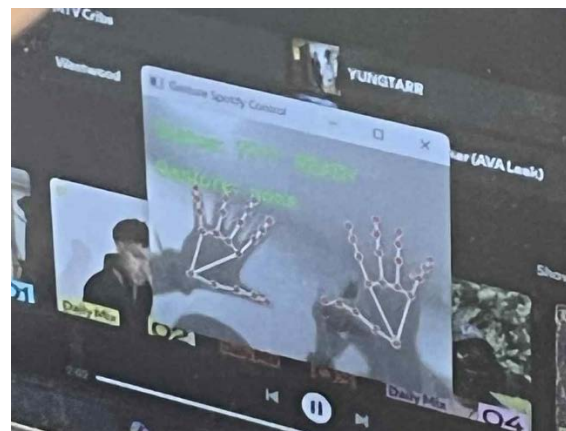
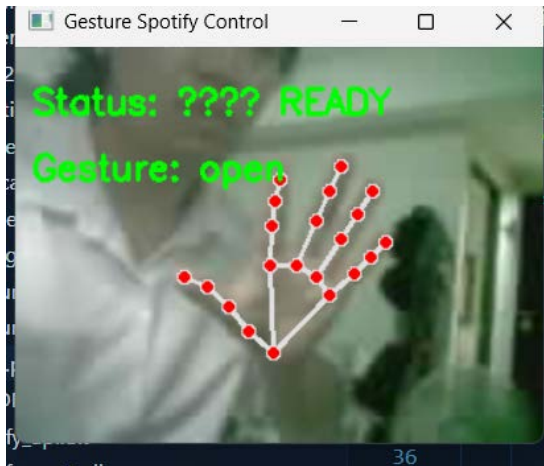
5. Photos & Evidence



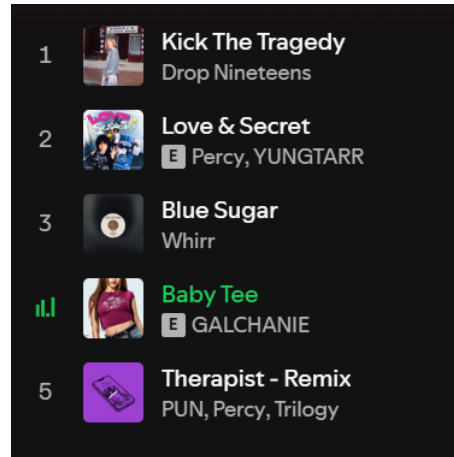
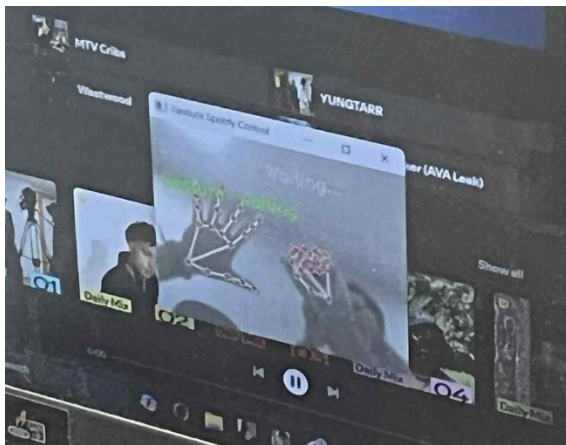
The overview of product, esp32 cam on it base and cardboard.



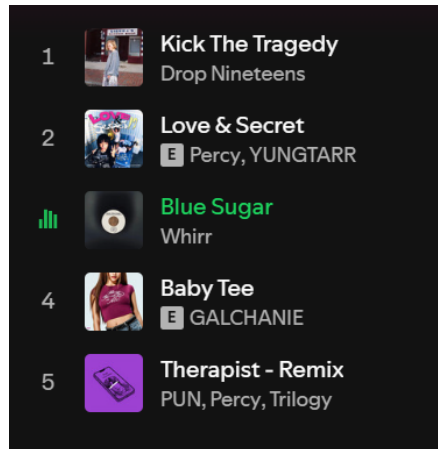
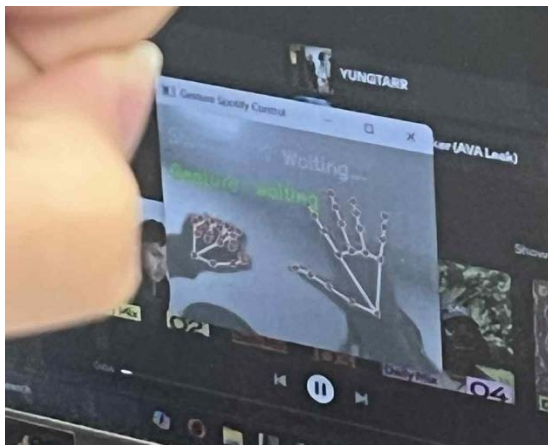
App page



1.open hand; ready (do not thinng)



2.left open right fist; skips song



3.right open left fist; previous song

6.Conclusion & Future Work

Conclusion

- Touchless Spotify control with ESP32-CAM works in real time on Windows.
- Preview UI reduces mistakes; caching reduces API calls.
- Initial results: ~19–20 FPS, ~340 ms median latency, ~94% accuracy in normal light.
- Limits: low light/complex background, same Wi-Fi, Spotify Premium needed.

Future Work

- Improve accuracy and stability (adaptive thresholds, smoothing, auto calibration).
- Add features: more gestures, multi-user switch, hold-to-accelerate volume/seek.
- Better UX: audio/visual confirmation, user profiles and settings UI.
- Performance: threading/async, auto resolution scaling.
- Security/packaging: move secrets to environment/.env, Windows installer/EXE.
- Testing/metrics: automated latency/accuracy logger, unit/integration tests.
- Platforms: Linux/RPi support, USB camera option.